CS 454 Project 1 - Write-Up

Jonathan Bird and Diego Delgado

For the first DFA, we used the function originalDFA() to construct a DFA for the language L, which reads strings using the alphabet {a, b, c, d}, where every substring of length six must contain all four letters. We realized we would only need to remember the last five characters because we were going to add another character to the end, making it length six. There was a check to see if the string contained all four letters, and if it did, it removed the oldest character and it transitioned to the state representing the remaining five characters. If the string was missing any required letter, it would then lead to a rejecting fail state. The program we implemented uses the addState() function and then addSink() to immediately handle any invalid transitions.

For the second DFA, we created a DFA that was just like the language for L except for the added language that required having “aa” as the two middle characters. This DFA also included 16 input symbols because each input symbol would represent a pair of letters from a, b, c, or d. We were stuck on this part of the project for a while because although it was similar to the first DFA, the use of many more possible DFA’s and pairing the characters together made it a little more difficult to understand how to implement the DFA. What we found would work best was taking two different pairs of the DFA. We would store the left and right sides of the DFA. The middle of the string “AA” was treated as a fixed character, and it did not need to be treated as an input symbol. To determine if the paired state was going to be accepted, it would check and see if the right and left sides were both valid and joined around the “aa”. The buildMiddleAADFA() would construct the second DFA, and the fasterPairedCount() counts the strings that were accepted. Eventually, when we got the code to work for the second DFA, we realized that it was super slow when the numbers got too big (around 300). We found a way to get around this super long runtime with our other algorithm. The original was representing around 1.8 million states, with there being 16 transitions per state. Instead, we used fasterPairedCount(), which uses arrays that have around 1400 entries, and we would only need to examine 4 transitions per entry. Since both halves are identical copies of the original DFA, we were able to count them separately and multiply their counts when combining them. We still have the old code in the LDoubleDFA.py file; it is just not being used in order to execute the file faster. Both methods lead to the same number, but the second one we came up with was much faster. Overall, we were able to improve the performance by avoiding calculations over every paired DFA state and instead by counting each half separately, then combining compatible four-character endings around the middle “aa”.